

MIAE 215 · Comprehensive Topic Guide (Part 3)

Expressions, Operators, Math Library Functions & Numerical Optimization

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Table of Contents

1. [Expressions, Operands & The Assignment Operator \(= \)](#)
2. [Sequential Memory Mutation vs. Simultaneous Algebraic Equations](#)
3. [Chained / Extended Assignment Mechanics](#)
4. [Arithmetic Operators & The Truncated Integer Division Trap](#)
5. [The Modulo Operator \(% \) & Discrete Engineering Algorithms](#)
6. [Prefix vs. Postfix Increment \(++x vs. x++ \) & Compound Assignment](#)
7. [The Operator Precedence & Associativity Hierarchy](#)
8. [Mixed-Type Expressions & Implicit Coercion Rules](#)
9. [The C++ Standard Math Library \(<cmath> \) & Robotics Trigonometry](#)
10. [Floating-Point Roundoff Noise & Safe Epsilon Comparison](#)
11. [Engineering Optimization Case Study: Numerical Grid-Search Peak Finding](#)

1. Expressions, Operands & The Assignment Operator (=)

In computer programming, an **operator** performs a specific mathematical or logical manipulation upon one or more data inputs (**operands**). An **expression** is any valid sequence of operators and operands that evaluates to a single value.

Part of $y = x + z * w$	Items	Role
Operators	<code>=</code> , <code>+</code> , <code>*</code>	The actions performed
Operands	<code>y</code> , <code>x</code> , <code>z</code> , <code>w</code> (and intermediate results)	The values acted on

The Imperative Assignment Operator (=)

In pure mathematics, the equals symbol ($=$) states a static identity: $x = y$ is mathematically equivalent to $y = x$. In C++, `=` is an **imperative memory copy instruction**:

$$\text{LHS (lvalue)} = \text{RHS (rvalue)};$$

- **lvalue (Left-Hand Side)**: An expression referring to a modifiable, named memory location in RAM.

- **rvalue (Right-Hand Side):** A value or evaluated expression that is computed first and then copied into the lvalue's memory address.

```
int i;  
i = 7 * 7; // (1) CPU computes 7 * 7 = 49 in register  
          // (2) Value 49 is written into memory allocated for variable i
```

2. Sequential Memory Mutation vs. Simultaneous Algebraic Equations

One of the most critical hurdles for engineering students transitioning from pure mathematics to computer science is understanding **sequential execution lifecycle**.

Consider the following valid C++ statement:

```
x = 2 * x + 1;
```

Math Interpretation:

$x = 2x + 1$

$-x = 1$

$x = -1$ (Simultaneous Eq.)

C++ Imperative Execution:

Step 1: Read current value of x from RAM (e.g., $x = 3$)

Step 2: ALU evaluates expression: $2 * (3) + 1 = 7$

Step 3: Write new value 7 back into variable x in RAM

- In algebra, you solve for a static unknown x .
- In C++, a program executes **sequentially line-by-line**. Each line mutates the current state of variables stored in RAM. The right-hand side is evaluated using the variable's *current* value, and the result overwrites the previous value.

3. Chained / Extended Assignment Mechanics

C++ allows multiple assignment operators to be chained together in a single statement:

```
double z1, z2, z3;  
z1 = z2 = z3 = 2.0 * 5.5; // All three variables become 11.0
```

Associativity: Right-to-Left

The assignment operator `=` has **right-to-left associativity**. The statement above executes in the following exact sequence:

1. `2.0 * 5.5` is evaluated to `11.0`.
2. `z3 = 11.0` executes first, storing `11.0` into `z3` and returning `11.0`.
3. `z2 = z3` executes second, storing `11.0` into `z2` and returning `11.0`.
4. `z1 = z2` executes last, storing `11.0` into `z1`.

Teacher Exam Warning / Pitfall:

Avoid chained assignments if different variables might require separate modifications later in the code. It is safer engineering practice to write:

```
double product = 2.0 * 5.5;
z1 = product;
z2 = product;
z3 = product;
```

4. Arithmetic Operators & The Truncated Integer Division Trap

C++ provides five basic arithmetic operators:

Operator	Operation	Example (int)	Example (double)
+	Addition	<code>7 + 3</code> $\implies$ 10	<code>7.0 + 3.0</code> $\implies$ 10.0
-	Subtraction	<code>7 - 3</code> $\implies$ 4	<code>7.0 - 3.0</code> $\implies$ 4.0
*	Multiplication	<code>7 * 3</code> $\implies$ 21	<code>7.0 * 3.0</code> $\implies$ 21.0
/	Division	<code>7 / 3</code> $\implies$ 2 (Truncated!)	<code>7.0 / 3.0</code> $\implies$ 2.333333...
%	Modulus (Remainder)	<code>7 % 3</code> $\implies$ 1	<i>Not defined for floating types</i>

The Cardinal Trap: Truncated Integer Division

When both operands of `/` are integers, the CPU performs **integer division**, permanently discarding any fractional remainder:

```
int i = 1 / 3;    // Evaluates to 0!
double x = 1 / 3; // Evaluates to 0, then implicitly widens to 0.0!
```

If an engineer writes:

```
double area = 1/2 * base * height; // BUG: 1/2 evaluates to 0! area is ALWAYS 0.0!
```

The Two Safe Solutions:

1. Use Floating-Point Literals:

```
double area = 1.0 / 2.0 * base * height; // Correct: 0.5 * base * height
```

2. Explicit Type Casting:

```
int a = 1, b = 2;
double area = (double)a / b * base * height; // Promotes to double division
```

5. The Modulo Operator (%) & Discrete Engineering Algorithms

The modulus operator (%) computes the integer remainder after division:

$$a = b \cdot q + r \quad \text{where} \quad 0 \leq r < |b|$$

```
int r = 7 % 3; // r = 1 (since 7 = 3 * 2 + 1)
```

Pattern A: Base-10 Digit Peeling (Digit Extraction)

In embedded systems, integers often need to be broken into individual decimal digits to drive 7-segment displays or format network packets without string conversions:

```
int b = 237;

int digit_units    = b % 10; // 237 % 10 = 7 (Extracts rightmost digit)
b = b / 10;         // 237 / 10 = 23 (Truncates rightmost digit)

int digit_tens     = b % 10; // 23 % 10 = 3
b = b / 10;        // 23 / 10 = 2

int digit_hundreds = b % 10; // 2 % 10 = 2
```

Pattern B: Circular Ring Buffers

To keep an index wrapping smoothly within an array of size N (e.g., a rotating sensor history buffer):

```
index = (index + 1) % BUFFER_SIZE; // Wraps from BUFFER_SIZE-1 back to 0
```

Pattern C: Alternating Series $(-1)^n$

```
// Generates +1 for even iterations, -1 for odd iterations:
int sign = (i % 2 == 0) ? 1 : -1;
```

6. Prefix vs. Postfix Increment (`++x` vs. `x++`) & Compound Assignment

Increment (`++`) and Decrement (`--`)

- `x++` (or `++x`): Adds 1 to `x` (equivalent to `x = x + 1`).
- `x--` (or `--x`): Subtracts 1 from `x` (equivalent to `x = x - 1`).

The Crucial Distinction: Prefix vs. Postfix

POSTFIX (`z = x++`):

1. Fetch current `x` value into `z`
2. Increment `x` in memory (+1)

Result: `z` gets old `x`; `x` is updated

PREFIX (`z = ++x`):

1. Increment `x` in memory (+1)
2. Fetch updated `x` value into `z`

Result: `z` and `x` both get new value

```
int x = 1, z;

z = x++; // Postfix: z = 1, x = 2
cout << "z = " << z << ", x = " << x << "\n"; // Prints: z = 1, x = 2

x = 1;
z = ++x; // Prefix: x becomes 2, then z = 2
cout << "z = " << z << ", x = " << x << "\n"; // Prints: z = 2, x = 2
```

Compound Assignment Operators

C++ provides shorthand compound operators: `+=`, `-=`, `*=`, `/=`.

- `x += 5.5;` is equivalent to `x = x + 5.5;`
- `x -= 2.0;` is equivalent to `x = x - 2.0;`

Teacher Exam Warning / Pitfall:

Avoid using `*=` and `/=` for unit conversions inside iterative loops!

```
// DANGEROUS CODE IN A SIMULATION LOOP:
for (int step = 0; step < 1000; step++) {
    velocity *= 0.277778; // ACCUMULATING BUG: velocity drops to 0.0 rapidly!
}
```

7. The Operator Precedence & Associativity Hierarchy

When an expression contains multiple operators, C++ evaluates them according to a strict mathematical hierarchy.

Rank (highest first)	Operators	Associativity
1	Parentheses ()	inside out
2	Postfix x++, x--	left to right
3	Unary / prefix ++x, --x, -x, (type) cast	right to left
4	Multiplicative *, /, %	left to right
5	Additive +, -	left to right
6	Relational <, <=, >, >=	left to right
7	Equality ==, !=	left to right
8	Logical AND &&	left to right
9	Logical OR	left to right
10	Assignment =, +=, -=, *=, /=	right to left (so a = b = c = 7 works)

Case Study: Rational Formula Evaluation

To evaluate the rational engineering formula:

$$y = \frac{ax^2 + bx + c}{dx + e}$$

In C++, you **must** use parentheses to enforce correct order:

```
// INCORRECT (Common beginner syntax error):
y = a*x*x + b*x + c / d*x + e; // Evaluates c / d first, then multiplies!

// CORRECT ENGINEERING FORMULATION:
y = (a*x*x + b*x + c) / (d*x + e);
```

8. Mixed-Type Expressions & Implicit Coercion Rules

When an operator links operands of different types, C++ applies **implicit type coercion**:

1. If either operand is `double`, the other is promoted to `double`.
2. Else if either operand is `float`, the other is promoted to `float`.
3. Else integer promotions are applied (`char` and `short` promote to `int`).

```
double d1 = 5.0, d2;
float f1 = 7.0f;
int i1 = 3;

d2 = d1 + f1 * i1;
// 1. (f1 * i1): i1 promotes to float; result is float 21.0f
// 2. (d1 + 21.0f): 21.0f promotes to double 21.0
// 3. d2 receives 5.0 + 21.0 = 26.0
```

9. The C++ Standard Math Library (`<cmath>`) & Robotics Trigonometry

The C++ standard math library provides hardware-accelerated floating-point routines. To use them, include `#include <cmath>`.

Key Functions in `<cmath>`

Function	Signature	Mathematical Operation	Critical Engineering Notes
<code>sin(x)</code>	<code>double sin(double x)</code>	$\sin(x)$	Argument x MUST be in radians ($\text{rad} = \text{deg} \times \frac{\pi}{180}$).
<code>cos(x)</code>	<code>double cos(double x)</code>	$\cos(x)$	Argument x MUST be in radians .
<code>tan(x)</code>	<code>double tan(double x)</code>	$\tan(x)$	Singularity at $x = \frac{\pi}{2}$ ($\pm 90^\circ$).
<code>atan(x)</code>	<code>double atan(double x)</code>	$\arctan(x)$	Returns angle in radians within range $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ ($[-90^\circ, +90^\circ]$).
<code>atan2(y, x)</code>	<code>double atan2(double y, double x)</code>	Four-quadrant $\arctan(y/x)$	Returns angle in radians within range $[-\pi, +\pi]$ ($[-180^\circ, +180^\circ]$).
<code>exp(x)</code>	<code>double exp(double x)</code>	e^x	Exponential function ($e \approx 2.7182818$).
<code>log(x)</code>	<code>double log(double x)</code>	$\ln(x)$ (Natural Log)	Domain restriction: $x > 0$. If $x \leq 0$, returns <code>nan</code> (Not a Number).
<code>log10(x)</code>	<code>double log10(double x)</code>	$\log_{10}(x)$	Base-10 logarithm. Domain restriction: $x > 0$.
<code>sqrt(x)</code>	<code>double sqrt(double x)</code>	$\sqrt{x}$	Domain restriction: $x \geq 0$. If $x < 0$, returns <code>nan</code> (no imaginary numbers in <code><cmath></code>).
<code>pow(x, y)</code>	<code>double pow(double x, double y)</code>	x^y	Computes general powers. For integer powers (x^2, x^3), use <code>x*x</code> or <code>x*x*x</code> for 10× faster speed.
<code>abs(x)</code> / <code>fabs(x)</code>	<code>double abs(double x)</code>	$\ x\ $	Absolute value for floating-point and integer numbers.

Why `atan2(y, x)` is Indispensable in Robotics & Dynamics

In mobile robotics, drone navigation, and kinematics, calculating orientation angle θ from Cartesian coordinates (x, y) using `atan(y / x)` causes catastrophic failure:

- $\frac{y}{x} = \frac{-1}{-1} = +1 \implies \arctan(1) = 45^\circ$ (Quadrant 1). But $(-1, -1)$ is in **Quadrant 3** (-135°)!
- Furthermore, if $x = 0$, `atan(y / x)` attempts division by zero!

The function `atan2(y, x)` inspects the signs of both y and x independently:

```
double x = -1.0, y = -1.0;
double theta = atan2(y, x); // Correctly returns -2.35619 rad (-135 degrees!)
```

10. Floating-Point Roundoff Noise & Safe Epsilon Comparison

Because floating-point numbers are represented in binary (base 2), transcendental mathematical identities do not evaluate to exact integers:

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    const double PI = 4.0 * atan(1.0);
    double val = cos(PI / 2.0); // Mathematically, cos(pi/2) = 0

    cout.precision(16);
    cout << "cos(PI / 2.0) = " << val << "\n";
    // PRINTS: 6.123233995736766e-17 (NOT EXACT ZERO!)
    return 0;
}
```

The Epsilon (ϵ) Comparison Rule

Because $6.12 \times 10^{-17} \neq 0.0$, a naive condition `if (cos(PI/2.0) == 0.0)` will evaluate to **FALSE**.

In engineering software, always test floating-point numbers using an absolute tolerance ϵ :

```
double eps = 1.0e-9; // Engineering precision threshold

if (abs(cos(PI / 2.0) - 0.0) < eps) {
    cout << "Value is physically zero within tolerance.\n";
}
```

11. Engineering Optimization Case Study: Numerical Grid-Search Peak Finding

In `MIAE_215_week2_lecture2_in_person/lecture_example2b` and `assignment1/question7.cpp`, Prof. Gordon introduces numerical grid-search optimization.

The Engineering Problem

Find the global maximum value and corresponding location $x_{\max}$ of the non-linear function:

$$f(x) = \sin(x^2 - 1.0)(x - 3.0) + \cos(x) \quad \text{over the domain } 0 \leq x \leq 10\pi$$

The idea: sample $f(x)$ at evenly spaced points $x = 0, \Delta x, 2\Delta x, \dots$ up to 10π , and keep the largest value seen so far (and where it occurred). A smaller step Δx finds the peak more accurately but needs more loop iterations.

C++ Numerical Grid-Search Implementation

```
#include <iostream>
#include <cstdio>
#include <cmath>

using namespace std;

int main() {
    const double PI = 4.0 * atan(1.0);
    double x_start = 0.0;
    double x_end = 10.0 * PI;
    double dx = 0.01; // Grid discretization step

    // Initialize maximum search at start point
    double x = x_start;
    double f = sin(x*x - 1.0) * (x - 3.0) + cos(x);
    double max_f = f;
    double max_x = x;

    // Iterative grid-search evaluation
    for (x = x_start; x <= x_end; x += dx) {
        f = sin(x*x - 1.0) * (x - 3.0) + cos(x);

        if (f > max_f) {
            max_f = f; // Record new highest peak
            max_x = x; // Record location of highest peak
        }
    }

    cout.precision(6);
    cout << "Global Maximum f(x) = " << max_f << "\n";
    cout << "Optimal Location x = " << max_x << " rad\n";

    return 0;
}
```

Numerical Grid-Search Performance & Discretization Trade-off

Discretization Step (dx)	Total Function Evaluations	Precision of $x_{\max}$	Execution Time
$dx = 0.1$	315 points	Low (± 0.05)	< 1 ms
$dx = 0.01$	3,142 points	Medium (± 0.005)	< 1 ms
$dx = 0.0001$	314,160 points	High (± 0.00005)	≈ 12 ms